
Painel de Compras CEDEP

Documentação técnica do protótipo — para desenvolvimento do sistema real

CEDEP Comércio Ltda

Diretoria de Compras e Logística

Setembro de 2026 — revisão 2 (perguntas em aberto resolvidas com Felipe)

PROTOTIPO.md — Painel de Compras CEDEP

Documento de referência técnica do protótipo `.jsx`. Descreve exatamente o que existe no código, sem completar lacunas nem propor melhorias. Onde algo é provisório ou inventado pra fins de demonstração, está marcado explicitamente.

Arquivo fonte único: `painel_cedep_prototipo.jsx` (React, ~3.660 linhas, um componente por tela + componentes compartilhados, tudo num arquivo só).

1. O que é, em cinco linhas

Painel de apoio à decisão de compra e precificação pra um distribuidor de lubrificantes e autopeças (CEDEP). Reúne, num só lugar, os alertas de estoque que hoje vêm de planilha (ruptura, sem giro, margem fora do alvo), a decisão de quanto comprar de cada produto, a decisão de que preço praticar (atacado e varejo, sob diferentes regimes fiscais), e o fluxo de salvar/orçar/fechar/exportar um pedido de compra pro Winthor. Existe em duas “visões” completas e paralelas — mobile e desktop — trocadas manualmente por um botão, não por responsividade de tela.

2. As telas

O app tem duas dimensões de navegação: - **area**: `painel` | `pedidos` | `pedidos_salvos` | `precificacao` — trocada pelo menu ☰ no cabeçalho. - **aba**: só existe dentro de `area === "painel"` — `alertas` | `monitoramento` | `entradas` — trocada pela barra inferior fixa (mobile) / não existe barra equivalente fixa no desktop, a troca de aba é a mesma barra reaproveitada.

Além disso, `skuAberto` é um estado à parte que, quando preenchido, **substitui a tela inteira** por `TelaDecisaoSKU` (Nível 2), sobre qualquer área/aba — é o “abrir o produto”.

Nenhuma URL muda. Não há rotas, não há back do navegador funcional, não há deep-link. Recarregar a página a qualquer momento volta pro estado inicial (`area="painel"`, `aba="alertas"`, mobile, nada salvo — ver seção 8).

2.1 Painel → Alertas

Componente: `TelaAlertas`. **Estados internos:** `ativos` (tipos de alerta ligados/desligados), `filtroFornecedor`, `filtroSecao`, `filtroLinha`, `filtroStatus` (nasce em `"Ativo"`), `cenarioSel` (nasce em `"st_valor"`).

O que a pessoa faz, em ordem: 1. Vê 3 KPIs (Alertas no filtro, Valor em risco, Ruptura urgente). 2. Filtra por Status / Departamento / Seção / Linha / Cenário de margem. 3. Liga/desliga tipos de alerta (múltipla escolha, botões-etiqueta) — sem nenhum ligado, a lista mostra **todo mundo que tem pelo menos 1 alerta**; com algum ligado, filtra só quem tem aquele(s) tipo(s). 4. Lista ordenada por um score de prioridade (ver seção 5). 5. Toca/clica numa linha → abre `TelaDecisaoSKU` daquele produto.

Estados possíveis: - **Vazio**: `listaFiltrada.length === 0` → mensagem “Nenhum produto nesse filtro agora.” Não há estado de “sem alerta nenhum no sistema” tratado à parte — cai no mesmo vazio. - **Carregando**: não existe. Todo dado é síncrono (array em memória), não há requisição. - **Erro**: não existe tratamento de erro nessa tela. - **Sem permissão**: não existe controle de permissão em nenhuma tela do protótipo. - **Filtro Status ≠ “Ativo”**: mostra uma nota de aviso abaixo do filtro explicando o porquê (mobile) — no desktop essa nota **não existe** (assimetria real, ver seção 8).

Botões e o que fazem: - Toggle de tipo de alerta: liga/desliga aquele id no array `ativos`, sem navegação. - Linha da tabela / card: `onAbrirDecisao(p)` → seta `skuAberto`. - Nenhum botão de “salvar” ou “confirmar” nessa tela — é só leitura + navegação. - Sair sem salvar: não se aplica, não há edição aqui.

2.2 Painel → Monitoramento

Componente: `TelaMonitoramento`. **Estados:** `periodo` (Dia/Semana/Mês/Ano, nasce em `"Mês"`), `offsetPeriodo` (nasce em `0` = período atual), `metrica` (faturamento/peso/litros/quantidade), `filtroFornecedor/Secao/Linha`, `filtroStatus` (nasce em `"Todos"` — diferente de Alertas), `porProduto` (toggle), `grupoSelecionado`.

O que a pessoa faz, em ordem: 1. Escolhe período (granularidade + navegação ←/→ ou calendário nativo tocando no rótulo). 2. Escolhe métrica (segmented control). 3. Filtra por Status/Departamento/Seção/Linha. 4. Vê o total consolidado (2 cartões: valor do período + comparativo com ano anterior). 5. Por padrão, vê uma quebra automática pela primeira dimensão ainda não fixada (Departamento → Seção → Linha → Categoria — ver regra completa na seção 5). 6. Pode alternar pra “Detalhar por produto”, que troca a tabela de dimensão agregada por uma lista de produtos individuais (ainda respeitando os filtros).

Estados possíveis: - **Vazio**: `listaAtual.length === 0` ou `produtosFiltrados.length === 0` → linha de tabela/mensagem “Nada nesse cruzamento de filtros.” - **Carregando / Erro / Sem permissão**: não existem, mesma razão da tela anterior. - Não há estado de “nenhum resultado pro período” tratado separadamente de “nenhum

resultado pro filtro” — os dados de `DADOS / TOP_PRODUTOS_GERAL` **não mudam com o período selecionado** (ver seção 8 — é o maior “de mentira” da tela).

Botões e o que fazem: - “Detalhar por produto”: toggle visual, não busca nada novo. - Select de grupo (quando `porProduto` ligado e existe dimensão fixada): filtra a lista de produtos por aquele valor específico da dimensão. - Linha da tabela de produto: **não é clicável** nessa tela — diferente de Alertas/Pedidos/ Precificação, aqui não existe `onAbrirDecisao`. É uma assimetria real do protótipo.

2.3 Painel → Entradas

Componente: `TelaEntradas`. **Estados:** `periodo` (Dia/Semana/Mês — **não tem “Ano”**, diferente de Monitoramento), `offsetPeriodo`, `fornecedor/secao/linha`, `status` (nasce em `“Todos”`), `busca`.

O que a pessoa faz, em ordem: 1. Escolhe período, filtra por Departamento/Seção/Linha/Status, busca por nome ou código. 2. Vê o valor total das entradas no período + filtro. 3. Vê a lista agrupada por “Quando” (Hoje/Ontem/ Essa semana/Esse mês — só os grupos que têm pelo menos 1 item aparecem).

Estados possíveis: - **Vazio:** `filtradas.length === 0` → “Nenhuma entrada nesse filtro.” - Mesma ausência de Carregando/Erro/Sem permissão das outras telas. - Igual a Monitoramento: a lista de entradas (`ENTRADAS`) é **estática**, não muda de verdade com o período selecionado — o filtro de período existe na interface mas não filtra o dado real (ver seção 8).

Botões: nenhum além dos filtros/seleção de período. Não é clicável pra abrir produto.

2.4 Pedidos

Componente: `TelaPedidos`. **Estados:** `confirmacao` (mensagem de sucesso, null por padrão), `filtroFornecedor/Secao/Linha`, `filtroStatus` (nasce `“Ativo”`), `ordenacao` (nasce `“cobertura”`), `busca`, `unidadeTotal` (nasce `“valor”`), `pedidos` (objeto `{codigo: quantidadeDigitada}`), é o “carrinho”.

O que a pessoa faz, em ordem: 1. Filtra e ordena a lista de produtos (Status/Departamento/Seção/Linha/ Ordenação/Busca). 2. Pra cada produto, vê estoque, pendência, histórico de venda (4 meses), cobertura atual vs. alvo, tendência, e — se o estoque não cobrir o alvo — um botão de “Sugestão” com a quantidade calculada (ver fórmula na seção 5). 3. Toca no botão de sugestão → preenche o campo de pedido daquele produto com o valor sugerido. Ou digita manualmente. 4. Vê o total do carrinho em 4 unidades alternáveis (R\$ / Peso / Litros / Qtd). 5. Toca “Salvar pedido(s)” → cria 1 pedido salvo **por fornecedor** presente no carrinho, limpa o carrinho, mostra banner de confirmação com link pra “Ir para Pedidos Salvos”.

Estados possíveis: - **Vazio:** `lista.length === 0` → “Nada nesse filtro.” Carrinho vazio → botão “Salvar pedido” desabilitado (`itensParaSalvar.length === 0`). - **Sucesso:** banner verde com contagem de pedidos salvos, dispensável (desktop) ou fixo até navegar (mobile). - Sem Carregando/Erro/Sem permissão.

Botões e o que fazem: - Botão de sugestão por linha: preenche só aquele campo, não salva nada ainda. - “Salvar pedido(s)”: cria os `pedidosSalvos`, zera `pedidos` (carrinho volta a `{}`). - Clique no nome do produto: abre `TelaDecisaoSKU`. - Sair sem salvar: **o carrinho digitado se perde** se a pessoa trocar de área/aba antes de clicar “Salvar pedido” — não há aviso de confirmação, nem rascunho automático.

2.5 Pedidos Salvos — lista

Componente: `TelaPedidosSalvos`. **Estados:** `impressao` (pedido em pré-visualização de PDF, null por padrão), `itemAberto` (pedido aberto em detalhe, null por padrão), `filtroFornecedor`, `busca`, `statusAtivos` (array, nasce [`“Rascunho”`, `“Orçamento Enviado”`] — “em andamento” já vem marcado).

Fluxo de status, sempre nessa ordem: **Rascunho** → **Orçamento Enviado** → **Fechado** → **Exportado**. Cada status tem um botão de avançar e (a partir de Orçamento Enviado) um botão de “desfazer, voltar pro status anterior”.

O que a pessoa faz, em ordem: 1. Filtra por Status (múltiplo)/Departamento/Busca. 2. Por pedido, vê fornecedor, data, nº de itens, status, valor total, e um conjunto de ações que muda conforme o status atual (ver tabela de botões abaixo). 3. Clica “Ver/editar” → abre `TelaDetalhePedidoSalvo`. 4. Clica “PDF”/“Orçamento em PDF” → abre `ComprovanteImpressao` (overlay de impressão). 5. Clica “Excel” → baixa `.xlsx` direto, sem preview. 6. Clica “Exportar Winthor” (só em Fechado) → baixa `.xlsx` no formato da rotina 220 e avança automaticamente pro status Exportado. 7. Clica “Excluir” → remove o pedido sem confirmação.

Status	Ações disponíveis
Rascunho	Ver/editar · PDF · Excel · Marcar enviado · Excluir
Orçamento Enviado	Ver/editar · Reenviar PDF · Marcar fechado · voltar p/ rascunho · Excluir
Fechado	Ver/editar (só leitura) · Exportar Winthor · voltar p/ orçamento enviado · Excluir
Exportado	Ver (só leitura) · Baixar novamente · voltar p/ fechado · Excluir

Estados possíveis: - **Vazio de verdade** (`pedidosSalvos.length === 0`): mensagem orientando a ir pra “Pedidos” e usar “Salvar pedido”. - **Vazio por filtro** (`pedidosSalvos.length > 0` e `listaFiltrada.length === 0`): mensagem diferente, “Nada nesse filtro.” - Sem Carregando/Erro/Sem permissão.

2.6 Pedidos Salvos → Detalhe / edição

Componente: `TelaDetalhePedidoSalvo`. Recebe `pedido`, `onVoltar`, `onSalvarEdicao`, `avancarStatus`. **Estados internos:** `itens` (cópia local editável do array de itens), `adicionando` (bool, troca a tela inteira por `TelaAdicionarProdutos`).

`editavel = status === "Rascunho" || status === "Orçamento Enviado"`. Em Fechado/Exportado os campos viram texto, sem input.

O que a pessoa faz, em ordem (quando editável): 1. Vê o controle de status (mesmos botões de avançar/desfazer da lista, replicados aqui — ver nota no código: “antes só existia na lista, dava pra ficar preso num status errado sem jeito de desfazer estando dentro do detalhe”). 2. Edita quantidade e/ou preço unitário item a item — total recalcula na hora. 3. Remove item (botão explícito) ou zera a quantidade e sai do campo (`onBlur`) — os dois caminhos removem a linha. 4. “+ Adicionar produtos ao pedido” → abre `TelaAdicionarProdutos`, filtrado pelo fornecedor deste pedido; ao confirmar, mescla no `itens` local (por código — quem já existia é atualizado, quem é novo é acrescentado). 5. “Salvar alterações” — só habilitado se `JSON.stringify(itens) !== JSON.stringify(pedido.itens)` (ou seja, comparação profunda simples, não por campo).

Estados possíveis: - **Vazio:** `itens.length === 0` → “Nenhum item — adicione produtos ou exclua o pedido.” - **Sair sem salvar:** clicar “Voltar pra lista” **descarta silenciosamente** qualquer edição feita em `itens` — não há confirmação, não há aviso. `onVoltar` só faz `setItemAberto(null)`, o estado local `itens` desse componente é destruído junto.

2.7 Pedidos Salvos → Adicionar produtos

Componente: `TelaAdicionarProdutos`. Recebe `fornecedor`, `itensExistentes`, `onConfirmar`, `onCancelar`.
Estados: `busca`, `quantidades` (pré-populado com as quantidades já digitadas dos itens existentes).

Mostra **só produtos do mesmo fornecedor** do pedido (regra: um pedido salvo é sempre de um fornecedor só, é exigência do formato Winthor). Cada linha mostra EST+PED, Cobertura/ Alvo e um botão de Sugestão (mesma fórmula de `calcularSugestaoPedido`), pra dar contexto de quanto pedir sem precisar sair da tela.

“Confirmar produtos” → recalcula `itensAtualizados` a partir de **todos os produtos do fornecedor com quantidade preenchida no momento** (não só os que mudaram) e devolve pro componente pai via `onConfirmar`. “Cancelar” descarta tudo digitado aqui.

2.8 Pedidos Salvos → Comprovante (impressão)

Componente: `ComprovanteImpressao`. Recebe `pedidoSalvo`, `onFechar`. Overlay de tela cheia (`fixed inset-0 z-50`) com um documento formatado (logo, tabela de itens, total, rodapé com endereço/telefone da CEDEP). Botão “Imprimir / Salvar PDF” chama `window.print()` nativo do navegador — **não gera PDF de verdade no protótipo**, depende do usuário escolher “Salvar como PDF” no diálogo de impressão do navegador. Botão “Fechar” volta pra lista sem gerar nada.

2.9 Precificação

Componente: `TelaPrecificacao`. **Estados:** `filtroFornecedor/Secao/Linha`, `filtroStatus` (nasce `"Ativo"`), `cenarioSel` (nasce `"st_valor"`), `ordenacao` (nasce `"margem"`), `busca`, `precosAtacado`, `precosVarejo` (objetos `{codigo: novoPreçoDigitado}`).

O que a pessoa faz, em ordem: 1. Filtra/ordena a lista (mesmos filtros de sempre + Cenário de margem). 2. Pra cada produto, vê Tributação, Custo, Valor NF, preço atual/MKP/margem/sugerido — tudo em atacado e varejo lado a lado — **todos recalculados conforme o cenário selecionado** (ver seção 5, é o ponto mais importante da tela). 3. Digita um novo preço à vista pra atacado e/ou varejo → vê ao vivo o MKP, a margem resultante, e o preço a prazo equivalente (com seu próprio MKP/margem). 4. Não existe botão de salvar nessa tela — o valor digitado fica só na tela, não é persistido em lugar nenhum (ver seção 8).

Estados possíveis: - **Vazio:** `lista.length === 0` → “Nada nesse filtro.” - Sem Carregando/Erro/Sem permissão.
- Varejo sem cenário “Oficial”: quando `cenarioSel === "oficial"` mas o produto está sendo visto em varejo, aparece nota “‘Oficial’ não existe no varejo — comparando com o cenário real do produto” (mobile) — no desktop essa mesma situação existe mas sem nota visível (assimetria, ver seção 8).

2.10 Nível 2 — Decisão do SKU

Componente: `TelaDecisaoSKU`. Recebe `produto`, `onVoltar`, `modoDesktop`. É aberta a partir de Alertas, Pedidos ou Precificação (qualquer lugar que tenha `onAbrirDecisao`). **Estados:** `pedido` (quantidade a comprar, texto livre), `altAtacado/altVarejo` (preços editáveis, nascem preenchidos com o preço “à vista” do cenário real do produto), `enviado` (bool), `janelaTendencia` (“3 meses” | “2 meses”), `cenarioSel` (nasce `"st_valor"`).

O que a pessoa faz, em ordem: 1. Vê contexto: estoque+pendente, prazo de entrega, cobertura atual vs. alvo, gráfico de venda dos últimos 4 meses + ano anterior, tendência, clientes atacado/varejo. 2. Vê avisos condicionais (sazonalidade, sucessão de produto, campanha ativa) — só aparecem se aquele campo estiver preenchido no produto. 3. Vê Tributação + Custo/Valor NF (mesmo padrão de Precificação). 4. Escolhe o Cenário de margem — isso muda **simultaneamente**: qual cenário fica em destaque no card de preço, quais os outros que precisam expandir “ver outros” pra aparecer, e a margem atual mostrada (ver seção 5). 5. Em cada bloco de preço (Atacado/Varejo): vê preço atual, MKP atual, margem atual, o cenário em destaque com o preço sugerido

pra bater a margem alvo, pode expandir “ver outros cenários”, e digita um preço final (nasce preenchido com a sugestão do cenário real) — vê MKP/margem resultante ao vivo, e o equivalente a prazo. 6. No card vermelho “Decisão de compra”: digita a quantidade a pedir, vê a cobertura projetada depois desse pedido, e clica “Enviar pra Cássio/Gabriela”.

Estados possíveis: - Não há “vazio” — a tela sempre recebe um produto específico. - “Enviado ✓”: depois de clicar o botão de envio, ele muda de cor (vermelho → verde) e o texto muda — mas **não desabilita**, dá pra clicar de novo, não há confirmação nem qualquer chamada real (ver seção 8). - Sem Carregando/Erro/Sem permissão.

Botão “Voltar”: `onVoltar()` → `setSkuAberto(null)`, volta pra tela de onde veio. Todo estado local desse componente (pedido digitado, preços alterados, “enviado”) **se perde**, mesmo que a pessoa volte e reabra o mesmo produto de novo.

3. Todo campo que aparece na tela

Organizado por tela. Fórmulas coladas literalmente do código onde marcado **CALCULADO**.

3.1 Alertas — cabeçalho / KPIs

rótulo na tela	nome no código	tipo	de onde vem
Alertas (filtro)	<code>totalAlertas</code>	número	CALCULADO — <code>baseFiltrada.filter(p => p.alertas.length > 0).length</code>
Valor em risco	<code>valorEmRisco</code>	R\$	CALCULADO — <code>comAlerta.reduce((s,p) => s + p.valorEstoque, 0)</code> , exibido / 1000 com 1 casa + “k”
Ruptura urgente	<code>rupturaUrgente</code>	número	CALCULADO — <code>baseFiltrada.filter(p => p.alertas.includes("ruptura")).length</code>

3.2 Alertas — tabela/lista de produtos

rótulo na tela	nome no código	tipo	de onde vem
Código	<code>p.codigo</code>	número	MOCK
Produto	<code>p.nome</code>	texto	MOCK
(fornecedor · seção, abaixo do nome)	<code>p.fornecedor</code> , <code>p.secao</code>	texto	MOCK
Classe	<code>p.classe</code>	“A” “B/C” “S/VEND”	MOCK
Alertas (badges)	<code>p.alertas</code>	array de ids	MOCK — lista de tipos já atribuídos ao produto, não recalculada em tela
— detalhe do badge (ex. “16d sem venda”)	<code>detalheAlerta(p, tipoId)</code>	texto	CALCULADO — switch por tipo: ruptura/baixo_giro/sem_giro usam <code>p.diasSemVenda</code> ; estoque_alto usa <code>p.mesesCobertura.toFixed(1)</code> ;

rótulo na tela	nome no código	tipo	de onde vem
			margem_baixa/margem_alta usam <code>p.margemAtacado*100</code> (⚠ usa o campo estático, não <code>margemAtualPorCenario</code> — não reage ao cenário)
Dias s/venda	<code>p.diasSemVenda</code>	número	MOCK
Cobertura	<code>p.mesesCobertura</code>	número (meses) ou null	MOCK
MKP AT	<code>info.mkpAtacado</code>	multiplicador	CALCULADO — <code>calcMKP(p.pvAtacado, custoPorCenario(p, cenarioSel))</code>
Margem AT	—	percentual	CALCULADO — <code>margemAtualPorCenario(p, cenarioSel, "atacado")*100</code>
Sugerido AT	<code>info.sugAtacado</code>	R\$	CALCULADO — preço à vista do cenário selecionado, via <code>calcularCenariosAtacado</code>
MKP VAR	<code>info.mkpVarejo</code>	multiplicador	CALCULADO — igual ao AT, trocando <code>p.pvVarejo</code>
Margem VAR	—	percentual	CALCULADO — <code>margemAtualPorCenario(p, cenarioSel, "varejo")*100</code>
Sugerido VAR	<code>info.sugVarejo</code>	R\$ ou “—”	CALCULADO — null se o cenário selecionado não existir em varejo (ex. “oficial”)
Valor em risco (coluna, só desktop)	<code>p.valorEstoque</code>	R\$	MOCK

3.3 Monitoramento

rótulo na tela	nome no código	tipo	de onde vem
Período (Dia/Semana/Mês/Ano)	<code>periodo</code>	texto	DIGITADO (seleção)
Métrica (Faturamento/Peso/Litros/Quantidade)	<code>metrica</code>	id	DIGITADO
[total do período]	<code>totalGeral</code>	conforme métrica	CALCULADO — <code>filtrados.reduce((s,d) => s + d[metrica], 0)</code>
[comparativo vs. ano anterior — rótulo]	<code>rotuloComparacaoAno(periodo, offset)</code>	texto	CALCULADO — string montada por regra (ver seção 5)

rótulo na tela	nome no código	tipo	de onde vem
[comparativo vs. ano anterior — %]	fixo <code>" +6,4% "</code>	percentual	MOCK — hardcoded, não calculado, não muda com filtro/período/produto
[Departamento/Seção/Linha/Categoria, na tabela quebrada]	<code>item.nome</code>	texto	DERIVADO — chave de agrupamento de DADOS , vem de agregarPor
[valor da métrica por grupo]	<code>item[metrica]</code>	conforme métrica	DERIVADO — soma de DADOS filtrado, agrupado por dimensão
vs. ano passado (por linha)	<code>item.crescimentoYoY</code>	percentual	MOCK (por linha de DADOS) → DERIVADO (média simples por grupo em agregarPor)
Proporção do total	—	percentual + barra	CALCULADO — <code>(item[metrica] / totalGeral) * 100</code>
# (ranking, modo “por produto”)	índice do array	número	CALCULADO — posição após <code>.sort()</code>
Código, Produto, Departamento, Seção (modo “por produto”)	<code>p.codigo/nome/fornecedor/secao</code>	—	MOCK (de TOP_PRODUTOS_GERAL)
[métrica] (modo “por produto”)	<code>p[metrica]</code>	conforme métrica	MOCK
vs. ano passado (modo “por produto”)	<code>p.crescimentoYoY</code>	percentual	MOCK

3.4 Entradas

rótulo na tela	nome no código	tipo	de onde vem
Código	<code>e.codigo</code>	número	MOCK
Produto	<code>e.nome</code>	texto	MOCK
Departamento	<code>e.fornecedor</code>	texto	MOCK
Seção	<code>e.secao</code>	texto	MOCK
Quando	<code>e.dia</code>	“Hoje” “Ontem” “Essa semana” “Esse mês”	MOCK — string fixa por item, não calculada a partir de uma data real
Quantidade	<code>e.qtd</code>	número	MOCK
Valor	<code>e.valor</code>	R\$	MOCK — no array atual, calculado no momento da definição do mock via

rótulo na tela	nome no código	tipo	de onde vem
			<code>round2(qtd * custo)</code> , mas isso acontece uma vez só, na carga do arquivo — não recalcula em tela
[total do período, topo]	<code>valorTotal</code>	R\$	<code>CALCULADO</code> — <code>filtradas.reduce((s,e) => s + e.valor, 0)</code>

3.5 Pedidos

rótulo na tela	nome no código	tipo	de onde vem
Código / cód. fábrica	<code>p.codigo</code> / <code>p.codFab</code>	número / texto	<code>MOCK</code>
Produto	<code>p.nome</code>	texto	<code>MOCK</code>
(ícone campanha/ sucessão, junto ao nome)	<code>p.campanhaAtiva</code> / <code>p.sucessaoPara</code>	—	<code>MOCK</code> — ícone só aparece <code>if</code> o campo não for null
Classe	<code>p.classe</code>	—	<code>MOCK</code>
Estoque	<code>p.estDisp</code>	número (ou caixas, se MASTER)	<code>MOCK</code>
Pend.	<code>p.estPend</code>	número	<code>MOCK</code>
EST+PED	—	número	<code>CALCULADO</code> — <code>p.estDisp</code> + <code>p.estPend</code>
Últ. Entrada	<code>p.ultimaEntrada</code> + <code>p.qtdUltimaEntrada</code>	data texto “dd/mm/aa” + número	<code>MOCK</code>
Atual / M-1 / M-2 / M-3	<code>p.vendaHistorico[0..3]</code>	números	<code>MOCK</code>
Média	—	número	<code>CALCULADO</code> — <code>hist.reduce((s,v)=>s+v, 0)/hist.length</code> (média simples dos 4 meses)
Últ. Saída	<code>p.ultimaSaida</code> + <code>p.qtdUltimaSaida</code>	data texto + número	<code>MOCK</code>
Clientes AT/VAR	<code>p.clientesAtacado</code> / <code>p.clientesVarejo</code>	números	<code>MOCK</code>
Cob./Alvo	<code>p.mesesCobertura</code> / <code>p.coberturaAlvo</code>	meses / meses	<code>MOCK</code> (os dois campos)
Tend.	—	percentual + seta	<code>CALCULADO</code> — <code>((hist[0]- hist[3])/hist[3])*100</code> se <code>hist[3] > 0</code> , senão <code>0</code> . Seta: <code>setaTendencia</code> (>3%=↑navy,

rótulo na tela	nome no código	tipo	de onde vem
			<-3%=>vermelho, senão → cinza)
Mínimo	<code>p.pedidoMinimo</code>	R\$ ou “—”	MOCK
Sugestão	—	número + unidade	CALCULADO — ver <code>calcularSugestaoPedido</code> na seção 5
Pedido	<code>pedidos[p.codigo]</code>	número	DIGITADO
Total em R\$/Peso/Litros/ Qtd	<code>totais.*</code>	conforme seleção	CALCULADO — soma de todos os itens do carrinho na unidade escolhida (fórmula completa na seção 5)

3.6 Pedidos Salvos — lista

rótulo na tela	nome no código	tipo	de onde vem
Fornecedor	<code>p.fornecedor</code>	texto	DERIVADO — vira o “dono” do pedido no momento de <code>salvarPedidos</code> , um por fornecedor no carrinho
Data	<code>p.dataCriacao</code>	texto “dd/mm/aa”	MOCK — hardcoded “28/08/26” sempre , não é a data real do dispositivo
Itens	<code>p.itens.length</code>	número	DERIVADO
Status	<code>p.status</code>	um de <code>STATUS_PEDIDO</code>	DIGITADO (via botões de avançar/voltar)
Valor	<code>p.valorTotal</code>	R\$	CALCULADO — soma de <code>it.valorItem</code> de todos os itens, recalculada a cada edição

3.7 Detalhe de Pedido Salvo — por item

rótulo na tela	nome no código	tipo	de onde vem
Nome do produto	<code>it.nome</code>	texto	DERIVADO — copiado de <code>PRODUTOS</code> no momento em que o item entrou no pedido
Código / cód. fornecedor / embalagem	<code>it.codigo</code> / <code>it.codFab</code> / <code>it.embalagem</code>	—	DERIVADO , mesma origem
Qtd. (caixas ou unid.)	<code>it.quantidadeDigitada</code>	número	DIGITADO

rótulo na tela	nome no código	tipo	de onde vem
Preço unit.	<code>it.precoUnitario</code>	R\$	DIGITADO (editável em tela) — nasce como <code>p.custoGerencial</code> no momento da criação do item
Total (por item)	<code>it.valorItem</code>	R\$	CALCULADO — <code>it.unidades * it.precoUnitario</code>
Total do pedido	<code>valorTotal</code>	R\$	CALCULADO — soma de <code>it.valorItem</code>

3.8 Precificação

rótulo na tela	nome no código	tipo	de onde vem
Código / Produto	<code>p.codigo</code> / <code>p.nome</code>	—	MOCK
Tributação (badge)	<code>labelTributacaoCurta(p.modalidade)</code> + “· Mono” condicional	texto	CALCULADO a partir de <code>p.modalidade</code> (MOCK) e <code>p.creditoPisCofins === 0</code> (MOCK)
Tributação (subtítulo, regime real)	<code>p.regimeFiscal</code>	texto	MOCK
Custo	<code>custoPorCenario(p, cenarioSel)</code>	R\$	CALCULADO (fórmula completa na seção 5) — se igual a <code>p.custoUltimaEntrada</code> , mostra 1 valor; se diferente, mostra “últ” e “ger” separados
Valor NF	<code>valorAntesDoCredito(custoCenario, p.creditoICMS, p.creditoPisCofins)</code>	R\$	CALCULADO
Atacado atual	<code>p.pvAtacado</code>	R\$	MOCK
MKP AT	—	multiplicador	CALCULADO — <code>calcMKP(p.pvAtacado, custoCenario)</code>
Margem AT	—	percentual	CALCULADO — <code>margemAtualPorCenario(p, cenarioSel, "atacado")</code>
Sugerido AT	<code>selAtacado.avista</code>	R\$	CALCULADO — via <code>calcularCenariosAtacado</code>
Novo preço AT (input)	<code>precosAtacado[p.codigo]</code>	R\$	DIGITADO

rótulo na tela	nome no código	tipo	de onde vem
— MKP/Margem novos (ao lado do input)	<code>mkpNovoAT</code> / <code>margemNovoAT</code>	multiplicador / percentual	<code>CALCULADO</code> — só aparece se o input tiver valor > 0
— a prazo (linha de baixo)	<code>prazoAT</code> / <code>mkpPrazoAT</code> / <code>margemPrazoAT</code>	R\$ / mult. / %	<code>CALCULADO</code> — <code>precoNovo * FATOR_PRAZO_ATACADO</code> e daí margem/MKP sobre esse valor
Varejo atual / MKP VAR / Margem VAR / Sugerido VAR / Novo preço VAR	—	—	mesma lógica do bloco Atacado, trocando as constantes de varejo

3.9 Nível 2 — Decisão do SKU

rótulo na tela	nome no código	tipo	de onde vem
Nome, código, fornecedor, cód. fábrica, classe	<code>produto.*</code>	—	<code>MOCK</code>
Estoque + Pendente	—	número	<code>CALCULADO</code> — <code>produto.estDisp + produto.estPend</code>
Prazo de entrega	<code>produto.prazoEntregaDias</code>	dias	<code>MOCK</code>
Cobertura atual	<code>coberturaAtual</code>	meses	<code>CALCULADO</code> — <code>produto.mediaJanela > 0 ? produto.estDisp / produto.mediaJanela : null</code>
Alvo p/ [fornecedor]	<code>produto.coberturaAlvo</code>	meses	<code>MOCK</code>
Venda (mini-gráfico)	<code>produto.vendaHistorico[0..3]</code> + <code>produto.vendaAnoPassado</code>	números	<code>MOCK</code>
Tend. [janela]	<code>tendenciaPeriodo</code>	percentual	<code>CALCULADO</code> — compara média dos primeiros N meses da janela com a média dos N meses seguintes (fórmula completa seção 5)
vs. ano passado	<code>tendenciaYoY</code>	percentual	<code>CALCULADO</code> — <code>((hist[0] - vendaAnoPassado) / vendaAnoPassado) * 100</code>
Clientes atacado / varejo	<code>produto.clientesAtacado</code> / <code>Varejo</code>	números	<code>MOCK</code>
Avisos (sazonalidade/sucessão/campanha)	<code>produto.sazonalidade</code> / <code>sucessaoPara</code> / <code>campanhaAtiva</code>	texto ou null	<code>MOCK</code> — cada um é uma faixa colorida condicional

rótulo na tela	nome no código	tipo	de onde vem
Múltiplo de compra	<code>produto.embalCompra</code>	número	<code>MOCK</code>
Pedido mínimo do fornecedor	<code>produto.pedidoMinimo</code>	R\$ ou ausente	<code>MOCK</code>
Tributação / Custo / Valor NF	—	—	mesma lógica de Precificação (seção 3.8), usando <code>custoPorCenario(produto, cenarioSel)</code>
Cenário de margem (select)	<code>cenarioSel</code>	id	<code>DIGITADO</code>
Preço atual (por bloco)	<code>precoAtualAtacado</code> / <code>Varejo</code>	R\$	<code>MOCK</code> — <code>produto.pvAtacado</code> / <code>pVarejo</code>
MKP atual	—	multiplicador	<code>CALCULADO</code> — <code>calcMKP(precoAtual, custoPorCenario(...))</code>
Margem atual ([nome do cenário])	—	percentual	<code>CALCULADO</code> — <code>margemAtualPorCenario(produto, cenarioSel, canal)</code>
Sugerido p/ meta (X%) — à vista / a prazo	<code>cenario.avista</code> / <code>cenario.prazo</code>	R\$	<code>CALCULADO</code> — via <code>calcularCenariosAtacado</code> / <code>Varejo</code>
Preço final decidido (à vista, input)	<code>valorDecisao</code> (<code>altAtacado</code> / <code>altVarejo</code>)	R\$	<code>DIGITADO</code> — nasce preenchido com <code>realAtacado.avista.toFixed(2)</code>
→ MKP / Margem (resultado do preço digitado)	<code>mkpNovo</code> / <code>margemNova</code>	mult. / %	<code>CALCULADO</code>
A prazo (equivalente do preço digitado)	<code>precoPrazo</code> + seu MKP/margem	R\$ + mult. + %	<code>CALCULADO</code> — <code>precoNovo</code> * <code>fatorPrazo</code>
Pedido (quantidade)	<code>pedido</code>	número	<code>DIGITADO</code>
= X unidades (se MASTER)	<code>pedidoUnidades</code>	número	<code>CALCULADO</code> — <code>pedidoNum</code> * <code>produto.embalCompra</code>
Cobertura após esse pedido	<code>coberturaAposPedido</code>	meses	<code>CALCULADO</code> — <code>(estDisp + estPend + pedidoUnidades) / mediaJanela</code> — corrigido em revisão (usava <code>pedidoNum</code> sem converter; ver seção 9)

4. Dados de exemplo, literais

4.1 PRODUTOS (8 SKUs — usado em Alertas, Pedidos, Precificação, Nível 2)

```

const PRODUTOS = [
  { codigo: 6771, nome: "YPF ELAION FS 540 5W40 1LT", codFab: "50563.32.3", fornecedor: "YPF",
    secas: "Lubrificante", linha: "Linha Leve", classe: "A", status: "Ativo", diasSemVenda: 2, mesesCobertura: 1.39, margemAtacado: 0.1049, margemVarejo: 0.295, valorEstoque: 62186, alertas: ["baixo_giro"], embalagem: "12X1 LT", embalCompra: 12, modalidade: "ST_SUBSTITUTO", regimeFiscal: "RE ST BA MVA 62,35% LUBRI (real)", icmsEfSaida: 0, icmsEfSemReducao: 0, pvAtacado: 37.65, pvVarejo: 50.15, creditoICMS: 0, creditoPisCofins: 0.0925, custoGerencial: 28.71, custoUltimaEntrada: 21.54, custoStValor: 29.436454453039996, mediaJanela: 1559, estDisp: 2166, estPend: 2166, margemAtacadoStValor: 0.08565526021142109, margemAtacadoOficial: 0.10491524540059773, margemAtacadoSemRed: 0.10491524540059773, margemVarejoStValor: 0.28053181549272194, margemVarejoSemRed: 0.29499120616814567, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [1152, 1747, 1487, 1443], vendaAnoPassado: 1300, clientesAtacado: 719, clientesVarejo: 18, prazoEntregaDias: 12, coberturaAlvo: 2.0, pedidoMinimo: 3000, sazonalidade: null, sucessoPara: null, campanhaAtiva: null, ultimaEntrada: "04/08/26", qtdUltimaEntrada: 1228, ultimaSaida: "29/08/26", qtdUltimaSaida: 42, litragemUnidade: 1, pesoUnidade: 0.854 },
  { codigo: 2549, nome: "INGRAX FORMULA SYNTH SL 10W40 1LT", codFab: null, fornecedor: "INGRAX", secas: "Lubrificante", linha: "Linha Leve", classe: "B/C", status: "Inativo", diasSemVenda: 4351, mesesCobertura: null, margemAtacado: 0.1154, margemVarejo: 0.1536, valorEstoque: 0, alertas: ["sem_giro"], embalagem: "12X1 LT", embalCompra: 12, modalidade: "ST_SUBSTITUTO", regimeFiscal: "RE ST BA MVA 62,35% LUBRI (real)", icmsEfSaida: 0, icmsEfSemReducao: 0, pvAtacado: 23.74, pvVarejo: 25.01, creditoICMS: 0, creditoPisCofins: 0.0925, custoGerencial: 17.86, custoUltimaEntrada: 11.33, custoStValor: 15.371984988767501, mediaJanela: 0, estDisp: 0, estPend: 0, margemAtacadoStValor: 0.10387308958961662, margemAtacadoOficial: 0.11539229019650382, margemAtacadoSemRed: 0.11539229019650382, margemVarejoStValor: 0.14264982594392245, margemVarejoSemRed: 0.15358408513654545, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [0, 0, 0, 0], vendaAnoPassado: 0, clientesAtacado: 0, clientesVarejo: 0, prazoEntregaDias: 15, coberturaAlvo: 2.0, pedidoMinimo: null, sazonalidade: null, sucessoPara: null, campanhaAtiva: null, ultimaEntrada: "07/08/13", qtdUltimaEntrada: 1, ultimaSaida: "02/10/14", qtdUltimaSaida: 1, litragemUnidade: 1, pesoUnidade: 0.8583 },
  { codigo: 7019, nome: "IPIRANGA F1 MASTER SIN SP 5W30 1LT", codFab: "31254353", fornecedor: "IPIRANGA", secas: "Lubrificante", linha: "Linha Leve", classe: "A", status: "Ativo", diasSemVenda: 16, mesesCobertura: 0.0, margemAtacado: 0.1293, margemVarejo: 0.332, valorEstoque: 0, alertas: ["ruptura"], embalagem: "24X1 LT", embalCompra: 24, modalidade: "ST_SUBSTITUTO", regimeFiscal: "RE ST BA MVA 62,35% LUBRI (real)", icmsEfSaida: 0, icmsEfSemReducao: 0, pvAtacado: 26.99, pvVarejo: 37.2, creditoICMS: 0, creditoPisCofins: 0.0925, custoGerencial: 19.92, custoUltimaEntrada: 14.95, custoStValor: 20.425983634955, mediaJanela: 19659, estDisp: 0, estPend: 0, margemAtacadoStValor: 0.11070179196165247, margemAtacadoOficial: 0.12933021262550948, margemAtacadoSemRed: 0.12933021262550948, margemVarejoStValor: 0.3184144184151882, margemVarejoSemRed: 0.3319300386764113, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [7343, 30340, 25130, 9985], vendaAnoPassado: 15000, clientesAtacado: 626, clientesVarejo: 23, prazoEntregaDias: 7, coberturaAlvo: 2.0, pedidoMinimo: 5000, sazonalidade: null, sucessoPara: null, campanhaAtiva: "Ruptura crítica - maior classe A do fornecedor, zero estoque", ultimaEntrada: "04/07/26", qtdUltimaEntrada: 24000, ultimaSaida: "15/08/26", qtdUltimaSaida: 850, litragemUnidade: 1, pesoUnidade: 1 },
  { codigo: 8761, nome: "CADILLAC AROMATICAR CARRO NOVO 1L", codFab: "7898578851451", fornecedor: "CADILLAC", secas: "Car Care", linha: "Geral", classe: "S/VEND", status: "Ativo", diasSemVenda: 11, mesesCobertura: null, margemAtacado: 0.149, margemVarejo: 0.2701, valorEstoque: 648, alertas: ["baixo_giro"], embalagem: "8X1", embalCompra: 8, modalidade: "NORMAL", regimeFiscal: "Monofásico (NCM 33074900, real)", icmsEfSaida: 0.1205892, icmsEfSemReducao: 0.205, pvAtacado: 50.94, pvVarejo: 63.67, creditoICMS: 0.07, creditoPisCofins: 0.0, custoGerencial: 30.87, custoUltimaEntrada: 30.87, custoStValor: 30.871683, mediaJanela: 0, estDisp: 21, estPend: 21, margemAtacadoStValor: 0.1489598939929328, margemAtacadoOficial: 0.2333706939929328, margemAtacadoSemRed: 0.1489598939929328, margemVarejoStValor: 0.2701298413695618, margemVarejoSemRed: 0.2701298413695618, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [1, 0, 0], vendaAnoPassado: 3, clientesAtacado: 0, clientesVarejo: 0, prazoEntregaDias: 6, coberturaAlvo: 2.0, pedidoMinimo: null, sazonalidade: null, sucessoPara: null, campanhaAtiva: null, ultimaEntrada: "14/08/26", qtdUltimaEntrada: 24, ultimaSaida: "20/08/26", qtdUltimaSaida: 1, litragemUnidade: 0, pesoUnidade: 0.125 },
  { codigo: 2720, nome: "CAR 80 LIMPA CARBURADOR 300 ML", codFab: "CAR 80 12", fornecedor: "CAR 80", secas: "Químico", linha: "Geral", classe: "B/C", status: "Ativo", diasSemVenda: 3, mesesCobertura: 1.06, margemAtacado: 0.0585, margemVarejo: 0.2005, valorEstoque: 117259, alertas: ["margem_baixa"], embalagem: "12X300 ML", embalCompra: 12, modalidade: "NORMAL", regimeFiscal: "Redução 41,176% c/PIS (real)", icmsEfSaida: 0.1205892, icmsEfSemReducao: 0.205, pvAta

```

```

cado: 25.39, pvVarejo: 33.19, creditoICMS: 0.07, creditoPisCofins: 0.086, custoGerencial:
15.34, custoUltimaEntrada: 15.34, custoStValor: 15.335025, mediaJanela: 7189, estDisp: 7644, es
tPend: 7644, margemAtacadoStValor: 0.05852107128790866, margemAtacadoOficial: 0.142931871287908
69, margemAtacadoSemRed: 0.05852107128790866, margemVarejoStValor: 0.20046248870141603, margemV
arejoSemRed: 0.20046248870141603, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico:
[5762, 7897, 3123, 6280], vendaAnoPassado: 6800, clientesAtacado: 523, clientesVarejo: 25, praz
oEntregaDias: 3, coberturaAlvo: 2.0, pedidoMinimo: null, sazonalidade: null, sucessoPara:
null, campanhaAtiva: null, ultimaEntrada: "27/08/26", qtdUltimaEntrada: 8400, ultimaSaida: "28/
08/26", qtdUltimaSaida: 310, litragemUnidade: 0.3, pesoUnidade: 0.3 },
{ codigo: 4717, nome: "3M LIMPA PARA-BRISA PAST 2X5G", codFab: "HB004463673", fornecedor: "3M
", secas: "Químico", linha: "Geral", classe: "S/VEND", status: "Inativo", diasSemVenda: 2977, m
esesCobertura: null, margemAtacado: 0.1815, margemVarejo: 0.3267, valorEstoque: 0, alertas: ["s
em_giro"], embalagem: "CX C/20 UN", embalCompra: 20, modalidade: "NORMAL", regimeFiscal: "Reduç
ão 41,176% c/PIS (real)", icmsEfSaida: 0.1205892, icmsEfSemReducao: 0.205, pvAtacado: 1.48, pvV
arejo: 2.12, creditoICMS: 0.07, creditoPisCofins: 0.086, custoGerencial: 0.71, custoUltimaEntra
da: 0.71, custoStValor: 0.711875, mediaJanela: 0, estDisp: 0, estPend: 0,
margemAtacadoStValor: 0.18150337837837838, margemAtacadoOficial: 0.26591417837837844, margemAta
cadoSemRed: 0.18150337837837838, margemVarejoStValor: 0.3267099056603774, margemVarejoSemRed: 0
.3267099056603774, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [0, 0, 0, 0], vendaA
noPassado: 0, clientesAtacado: 0, clientesVarejo: 0, prazoEntregaDias: 10, coberturaAlvo: 2.0,
pedidoMinimo: null, sazonalidade: null, sucessoPara: null, campanhaAtiva: null,
ultimaEntrada: "16/10/17", qtdUltimaEntrada: 2000, ultimaSaida: "07/07/18", qtdUltimaSaida:
50, litragemUnidade: 0, pesoUnidade: 0.05 },
{ codigo: 574, nome: "TF PSL 47 FILT OLEO FIAT 147/UNO/TEMPRA", codFab: "PSL47", fornecedor:
"TECFIL", secas: "Filtro", linha: "Linha Leve", classe: "B/C", status: "Ativo", diasSemVenda:
3, mesesCobertura: 2.88, margemAtacado: 0.2054, margemVarejo: 0.4308, valorEstoque: 5473, alert
as: [], embalagem: "1X1", embalCompra: 12, modalidade: "ST_SUBSTITUTO", regimeFiscal:
"RE ST BA MVA 71,78% s/PIS (real)", icmsEfSaida: 0, icmsEfSemReducao: 0, pvAtacado: 16.77, pvVa
rejo: 23.47, creditoICMS: 0.0665, creditoPisCofins: 0.0, custoGerencial: 12.41, custoUltimaEntra
da: 9.69, custoStValor: 12.654340885616998, mediaJanela: 299, estDisp: 441, estPend: 861, marg
emAtacadoStValor: 0.20541795553864053, margemAtacadoOficial: 0.21992195860912356, margemAtacado
SemRed: 0.21992195860912356, margemVarejoStValor: 0.42082910585355776, margemVarejoSemRed: 0.43
119263936408186, margemAlvo: 0.2, margemAlvoVarejo: 0.2, vendaHistorico: [269, 357, 331, 258],
vendaAnoPassado: 290, clientesAtacado: 98, clientesVarejo: 7, prazoEntregaDias: 4, coberturaAlv
o: 2.0, pedidoMinimo: null, sazonalidade: null, sucessoPara: null, campanhaAtiva: null, ultima
Entrada: "13/07/26", qtdUltimaEntrada: 240, ultimaSaida: "28/08/26", qtdUltimaSaida: 18, litrag
emUnidade: 0, pesoUnidade: 0.492 },
{ codigo: 8825, nome: "ROBUST PNEU MOTO 60/100-17 BILIS TT", codFab: "403823", fornecedor: "R
OBUST", secas: "Pneu", linha: "Moto", classe: "S/VEND", status: "Ativo", diasSemVenda: 3, meses
Cobertura: null, margemAtacado: 0.1984, margemVarejo: 0.3011, valorEstoque: 5185, alertas: ["se
m_giro"], embalagem: "1X1", embalCompra: 1, modalidade: "ST_RECOLHIDO", regimeFiscal: "ST já
recolhido antes - BA (real)", icmsEfSaida: 0, icmsEfSemReducao: 0, pvAtacado: 111.6, pvVarejo:
129.0, creditoICMS: 0, creditoPisCofins: 0, custoGerencial: 85, custoUltimaEntrada: 85, custoSt
Valor: 85, mediaJanela: 0, estDisp: 61, estPend: 61, margemAtacadoStValor: 0.1983512544802867,
margemAtacadoOficial: 0.1983512544802867, margemAtacadoSemRed: 0.1983512544802867, margemVarejo
StValor: 0.3010852713178295, margemVarejoSemRed: 0.3010852713178295, margemAlvo: 0.2, margemAlv
oVarejo: 0.2, vendaHistorico: [21, 0, 0, 0], vendaAnoPassado: 15, clientesAtacado: 0, clientesV
arejo: 0, prazoEntregaDias: 20, coberturaAlvo: 2.0, pedidoMinimo: null, sazonalidade: null, suc
essaoPara: null, campanhaAtiva: null, ultimaEntrada: "18/08/26", qtdUltimaEntrada: 100, ultimaS
aida: "28/08/26", qtdUltimaSaida: 21, litragemUnidade: 0, pesoUnidade: 3.5 },
];

```

Nota de proveniência: esses 8 registros não são aleatórios — foram extraídos e validados linha a linha contra a planilha real `MODELO_COMPRAS_CEDep_V10.xlsx` (aba `pedido`), incluindo os campos fiscais (`modalidade`, `icmsEfSaida`, `icmsEfSemReducao`, `creditoICMS`, `creditoPisCofins`, `custoStValor`, as 5 variantes de margem). O TI pode usar esses 8 códigos (6771, 2549, 7019, 8761, 2720, 4717, 574, 8825) como casos de teste de regressão contra o sistema real, já que os números batem.

4.2 **DADOS** (26 linhas — usado só em Monitoramento, agregado por fornecedor/seção/linha/categoria)

```

const DADOS = [
  { fornecedor: "YPF", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: 180000, peso: 15000, quantidade: 4200, litros: 4200, crescimentoYoY: 0.12, status: "Ativo" },
  { fornecedor: "YPF", secao: "Lubrificante", categoria: "Motor", linha: "Moto", faturamento: 45000, peso: 2200, quantidade: 1800, litros: 1800, crescimentoYoY: 0.08, status: "Ativo" },
  { fornecedor: "YPF", secao: "Lubrificante", categoria: "Câmbio/Transmissão", linha: "Industrial", faturamento: 95000, peso: 7000, quantidade: 1800, litros: 1800, crescimentoYoY: 0.05, status: "Ativo" },
  { fornecedor: "YPF", secao: "Lubrificante", categoria: "Diferencial", linha: "Pesado", faturamento: 62000, peso: 8500, quantidade: 900, litros: 900, crescimentoYoY: 0.03, status: "Ativo" },
  { fornecedor: "YPF", secao: "Lubrificante", categoria: "Graxa", linha: "Geral", faturamento: 30000, peso: 5500, quantidade: 1100, litros: 0, crescimentoYoY: 0.02, status: "Ativo" },
  { fornecedor: "TECFIL", secao: "Filtro", categoria: "Óleo", linha: "Linha Leve", faturamento: 40000, peso: 1400, quantidade: 1100, litros: 0, crescimentoYoY: -0.15, status: "Ativo" },
  { fornecedor: "TECFIL", secao: "Filtro", categoria: "Ar", linha: "Linha Leve", faturamento: 28000, peso: 1300, quantidade: 900, litros: 0, crescimentoYoY: -0.1, status: "Ativo" },
  { fornecedor: "TECFIL", secao: "Filtro", categoria: "Combustível", linha: "Pesado", faturamento: 22000, peso: 1900, quantidade: 1400, litros: 0, crescimentoYoY: -0.12, status: "Ativo" },
  { fornecedor: "TECFIL", secao: "Filtro", categoria: "Cabine", linha: "Linha Leve", faturamento: 8000, peso: 1500, quantidade: 800, litros: 0, crescimentoYoY: -0.08, status: "Ativo" },
  { fornecedor: "CAR 80", secao: "Químico", categoria: "Limpeza de motor", linha: "Geral", faturamento: 25000, peso: 1000, quantidade: 1900, litros: 0, crescimentoYoY: 0.06, status: "Ativo" },
  { fornecedor: "OUTROS", secao: "Fluido Automotivo", categoria: "Freio", linha: "Geral", faturamento: 40000, peso: 1800, quantidade: 900, litros: 900, crescimentoYoY: -0.04, status: "Ativo" },
  { fornecedor: "OUTROS", secao: "Fluido Automotivo", categoria: "Arrefecimento", linha: "Geral", faturamento: 33000, peso: 3400, quantidade: 800, litros: 800, crescimentoYoY: 0.02, status: "Ativo" },
  { fornecedor: "OUTROS", secao: "Químico", categoria: "Penetrante/spray", linha: "Geral", faturamento: 18000, peso: 700, quantidade: 1400, litros: 0, crescimentoYoY: -0.09, status: "Ativo" },
  { fornecedor: "OUTROS", secao: "Químico", categoria: "Aditivo de combustível", linha: "Geral", faturamento: 9000, peso: 600, quantidade: 800, litros: 0, crescimentoYoY: -0.06, status: "Ativo" },
  { fornecedor: "OUTROS", secao: "Fluido Automotivo", categoria: "Arla", linha: "Pesado", faturamento: 15000, peso: 900, quantidade: 400, litros: 400, crescimentoYoY: 0.05, status: "Ativo" },
  { fornecedor: "ROBUST", secao: "Pneu", categoria: null, linha: "Moto", faturamento: 76000, peso: 12800, quantidade: 620, litros: 0, crescimentoYoY: -0.05, status: "Ativo" },
  { fornecedor: "IPIRANGA", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: 198000, peso: 19659, quantidade: 19659, litros: 19659, crescimentoYoY: 0.31, status: "Ativo" },
  { fornecedor: "INGRAX", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: 41500, peso: 2900, quantidade: 890, litros: 890, crescimentoYoY: 0.18, status: "Ativo" },
  { fornecedor: "INGRAX", secao: "Lubrificante", categoria: "Motor", linha: "Pesado", faturamento: 26500, peso: 2200, quantidade: 500, litros: 500, crescimentoYoY: 0.15, status: "Ativo" },
  { fornecedor: "CADILLAC", secao: "Car Care", categoria: null, linha: "Geral", faturamento: 41000, peso: 1200, quantidade: 3100, litros: 3100, crescimentoYoY: -0.12, status: "Ativo" },
  { fornecedor: "3M", secao: "Químico", categoria: "Limpeza de motor", linha: "Geral", faturamento: 22300, peso: 410, quantidade: 2100, litros: 0, crescimentoYoY: 0.1, status: "Ativo" },
  { fornecedor: "3M", secao: "Suprimento de Oficina", categoria: null, linha: "Geral", faturamento: 10700, peso: 200, quantidade: 300, litros: 0, crescimentoYoY: 0.05, status: "Ativo" },
  { fornecedor: "MANN", secao: "Filtro", categoria: "Ar", linha: "Pesado", faturamento: 30000, peso: 1200, quantidade: 400, litros: 0, crescimentoYoY: -0.03, status: "Ativo" },
  { fornecedor: "MANN", secao: "Filtro", categoria: "Óleo", linha: "Pesado", faturamento: 22000, peso: 900, quantidade: 400, litros: 0, crescimentoYoY: -0.02, status: "Ativo" },
  { fornecedor: "WEGA", secao: "Filtro", categoria: "Combustível", linha: "Linha Leve", faturamento: 18000, peso: 700, quantidade: 900, litros: 0, crescimentoYoY: 0.04, status: "Ativo" },

```

```
{ fornecedor: "WEGA", secao: "Filtro", categoria: "Óleo", linha: "Linha Leve", faturamento: 20000, peso: 800, quantidade: 700, litros: 0, crescimentoYoY: 0.06, status: "Ativo" },
{ fornecedor: "ROBUST", secao: "Pneu", categoria: null, linha: "Moto", faturamento: 6200, peso: 780, quantidade: 40, litros: 0, crescimentoYoY: -0.7, status: "Inativo" },
];
```

Importante: **DADOS** é uma estrutura à parte de **PRODUTOS** — não é a soma dos 8 SKUs de **PRODUTOS**, é um conjunto ilustrativo próprio, com fornecedores que nem existem em **PRODUTOS** (MANN, WEGA, OUTROS). Seção/Categoria/Linha aqui são explicitamente marcadas no comentário do código como “ainda ilustrativas — não são dado real do Winthor”.

4.3 **TOP_PRODUTOS_GERAL** (8 linhas — usado só em Monitoramento, modo “Detalhar por produto”)

```
const TOP_PRODUTOS_GERAL = [
  { codigo: 6771, nome: "YPF ELAION FS 540 5W40 1LT", fornecedor: "YPF", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: round2(1152 * 37.65), peso: round2(1152 * 0.854), litros: 1152, quantidade: 1152, crescimentoYoY: 0.20, status: "Ativo" },
  { codigo: 2549, nome: "INGRAX FORMULA SYNTH SL 10W40 1LT", fornecedor: "INGRAX", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: 0, peso: 0, litros: 0, quantidade: 0, crescimentoYoY: 0, status: "Inativo" },
  { codigo: 7019, nome: "IPIRANGA F1 MASTER SIN SP 5W30 1LT", fornecedor: "IPIRANGA", secao: "Lubrificante", categoria: "Motor", linha: "Linha Leve", faturamento: round2(7343 * 26.99), peso: round2(7343 * 1), litros: 7343, quantidade: 7343, crescimentoYoY: 0.31, status: "Ativo" },
  { codigo: 8761, nome: "CADILLAC AROMATICAR CARRO NOVO 1L", fornecedor: "CADILLAC", secao: "Car Care", categoria: null, linha: "Geral", faturamento: round2(1 * 50.94), peso: round2(1 * 0.125), litros: 0, quantidade: 1, crescimentoYoY: -0.67, status: "Ativo" },
  { codigo: 2720, nome: "CAR 80 LIMPA CARBURADOR 300 ML", fornecedor: "CAR 80", secao: "Químico", categoria: "Limpeza de motor", linha: "Geral", faturamento: round2(5762 * 25.39), peso: round2(5762 * 0.3), litros: round2(5762 * 0.3), quantidade: 5762, crescimentoYoY: -0.152, status: "Ativo" },
  { codigo: 4717, nome: "3M LIMPA PARA-BRISA PAST 2X5G", fornecedor: "3M", secao: "Químico", categoria: "Limpeza de motor", linha: "Geral", faturamento: 0, peso: 0, litros: 0, quantidade: 0, crescimentoYoY: 0, status: "Inativo" },
  { codigo: 574, nome: "TF PSL 47 FILT OLEO FIAT 147/UNO/TEMPRA", fornecedor: "TECFIL", secao: "Filtro", categoria: "Óleo", linha: "Linha Leve", faturamento: round2(269 * 16.77), peso: round2(269 * 0.492), litros: 0, quantidade: 269, crescimentoYoY: -0.072, status: "Ativo" },
  { codigo: 8825, nome: "ROBUST PNEU MOTO 60/100-17 BILIS TT", fornecedor: "ROBUST", secao: "Pneu", categoria: null, linha: "Moto", faturamento: round2(21 * 111.6), peso: round2(21 * 3.5), litros: 0, quantidade: 21, crescimentoYoY: 0.40, status: "Ativo" },
];
```

faturamento/peso/litros aqui usam os mesmos 8 códigos de **PRODUTOS**, mas é um array **duplicado e independente** — se um número mudar em **PRODUTOS**, não muda aqui automaticamente. **round2(v) = Math.round(v * 100) / 100**.

4.4 ENTRADAS (6 linhas — usado só na tela Entradas)

```
const ENTRADAS = [
  { codigo: 2720, nome: "CAR 80 LIMPA CARBURADOR 300 ML", fornecedor: "CAR 80", secao: "Químico", linha: "Geral", status: "Ativo", qtd: 8400, valor: round2(8400 * 15.34), dia: "Ontem" },
  { codigo: 8825, nome: "ROBUST PNEU MOTO 60/100-17 BILIS TT", fornecedor: "ROBUST", secao: "Pneu", linha: "Moto", status: "Ativo", qtd: 100, valor: round2(100 * 85), dia: "Essa semana" },
  { codigo: 8761, nome: "CADILLAC AROMATICAR CARRO NOVO 1L", fornecedor: "CADILLAC", secao: "Car Care", linha: "Geral", status: "Ativo", qtd: 24, valor: round2(24 * 30.87), dia: "Esse mês" },
  { codigo: 6771, nome: "YPF ELAION FS 540 5W40 1LT", fornecedor: "YPF", secao: "Lubrificante", linha: "Linha Leve", status: "Ativo", qtd: 1228, valor: round2(1228 * 21.54), dia: "Esse mês" },
  { codigo: 574, nome: "TF PSL 47 FILT OLEO FIAT 147/UNO/TEMPRA", fornecedor: "TECFIL", secao: "Filtro", linha: "Linha Leve", status: "Ativo", qtd: 240, valor: round2(240 * 9.69), dia: "Esse mês" },
  { codigo: 7019, nome: "IPIRANGA F1 MASTER SIN SP 5W30 1LT", fornecedor: "IPIRANGA", secao: "Lubrificante", linha: "Linha Leve", status: "Ativo", qtd: 24000, valor: round2(24000 * 14.95), dia: "Esse mês" },
];
```

4.5 TIPOS (os 6 tipos de alerta possíveis)

```
const TIPOS = [
  { id: "ruptura", label: "Ruptura", icon: PackageX, cor: RED },
  { id: "sem_giro", label: "Sem giro", icon: Boxes, cor: "#8A8578" },
  { id: "baixo_giro", label: "Baixo giro", icon: PackageSearch, cor: "#B98A2E" },
  { id: "estoque_alto", label: "Estoque alto", icon: Boxes, cor: "#B98A2E" },
  { id: "margem_baixa", label: "Margem baixa", icon: TrendingDown, cor: RED },
  { id: "margem_alta", label: "Margem alta", icon: TrendingUp, cor: NAVY },
];
```

Nenhum dos 8 produtos de **PRODUTOS** usa **estoque_alto** ou **margem_alta** no dado de exemplo atual — os dois tipos existem no sistema de alertas (filtro, cores, ícones) mas não têm nenhum caso ilustrado.

4.6 Parâmetros fiscais fixos (constantes, não por produto)

```
const FATOR_PRAZO_ATACADO = 1.0317;
const FATOR_PRAZO_VAREJO = 1.086435;
const COMISSAO = 0.04;
```

5. Regras de negócio embutidas

Cada uma com local no código, o que faz, e o número/limite usado.

Fórmula de margem real — `calcMargemReal`, linha ~123. `margem = 1 - PIS/COFINS(saída) - 0,04 (COMISSÃO) - ICMS_efetivo(saída, do cenário) - custo/preço`. Validada contra 7 casos reais da planilha V10 (diferença só de arredondamento na 4ª casa decimal). Constante de comissão fixa em 4% pra qualquer produto/fornecedor — não há campo que a diferencie.

MKP (markup) — `calcMKP`, linha ~113. $MKP = \text{preço} \div \text{custo}$, multiplicador puro (ex. 1,35), não percentual. Constante em qualquer cenário fiscal — **não** muda com o cenário selecionado (diferente de custo e margem, que mudam).

Custo por cenário — `custoPorCenario`, linha ~145. Regra descoberta comparando dado real (não estava no protótipo original, foi corrigida em sessão de revisão): - Cenário “ST s/Valor” → usa `custoStValor`. - Cenário “Oficial” → usa `custoGerencial` (que já é, por definição, o custo no regime Oficial, incluindo ajustes específicos de fornecedor tipo o “custo adicional imagem” do Ingrax). - Cenário “Sem Redução” → **depende da modalidade do produto**: se `ST_SUBSTITUTO`, usa o mesmo valor de `custoGerencial` (Oficial); se `NORMAL` (ou qualquer outra), usa `custoStValor`.

Explicação fiscal, confirmada com Felipe: produtos com ST (Substituição Tributária) não recebem redução de alíquota de ICMS — só os produtos em regime Normal recebem. É por isso que, pra `ST_SUBSTITUTO`, “Oficial” e “Sem Redução” sempre coincidem (não existe uma versão “reduzida” separada pra esse regime — logo as duas leituras colapsam no mesmo valor); e pra Normal, quem coincide é “ST s/Valor” com “Sem Redução” (o produto Normal não tem cálculo de ST pra começo de conversa, então “ST s/Valor” e “Sem Redução” acabam descrevendo a mesma situação de fato). A divergência real que existe pra `ST_SUBSTITUTO` entre “ST s/Valor” e “Oficial” não vem de redução de ICMS — vem de outra parte do cálculo de custo (a base usada no MVA/ST), que este protótipo não desmembra em detalhe.

Cenários de sugestão de preço — `calcularCenariosAtacado`/`calcularCenariosVarejo`, linhas 85–104. Resolve o preço à vista que bate a margem alvo, isolando `preço` na mesma fórmula de margem: $\text{preço} = \text{custo} / (1 - \text{PIS/COFINS} - 0,04 - \text{ICMS}_{\text{ef}}(\text{cenário}) - \text{margemAlvo})$. Varejo **não tem cenário “Oficial”** — só Atacado, porque a redução de base é “exclusiva filiais 02/09” (comentário no código, não desenvolvido em nenhuma outra parte do protótipo — é uma regra de negócio citada mas não implementada em detalhe). `real: true` num cenário significa “é o regime que de fato se aplica a esse produto hoje”: `ST s/Valor` é `real` quando `modalidade === "ST_SUBSTITUTO"`; `Sem Redução` é `real` quando `modalidade === "NORMAL"`. Nenhum cenário nunca é marcado `real: true` pra `ST_RECOLHIDO` — cai no fallback (primeiro item do array) sempre que precisa de um “cenário real” e a modalidade é essa.

Fator a prazo — constantes fixas, `1,0317` (atacado) e `1,086435` (varejo). Multiplica o preço à vista pra chegar no preço a prazo, em qualquer lugar que mostre “a prazo” no protótipo. **Origem confirmada com Felipe**: são valores pré-definidos pela diretoria há tempos, já configurados na tabela de vendas do próprio Winthor — variam por região de venda (atacado/varejo) e pelo prazo (à vista/a prazo). Não são um cálculo (juros, etc.) feito pelo protótipo; são dados de entrada que já vêm prontos de outro lugar.

Sugestão de pedido — `calcularSugestaoPedido`, linha 2346.

```
mediaVenda = média dos 4 meses de vendaHistorico
necessario = coberturaAlvo * mediaVenda - (estDisp + estPend)
se necessario <= 0 ou mediaVenda === 0 → sugestão = 0
se fornecedor é MASTER → arredonda pra cima em caixas fechadas (embaCompra)
senão → arredonda pra cima em unidades
```

“MASTER” = lista fixa de 5 fornecedores: `["YPF", "INGRAX", "PETRONAS", "KOUBE", "VALVOLINE"]` (`isMasterFornecedor`, linha 1310) — repetida **literalmente 6 vezes** em pontos diferentes do código como array inline, além da função nomeada; não há um único ponto de verdade consistentemente usado (ver seção 9).

Cobertura crítica (visual) — usada em `TelaPedidos`, `TelaDecisaoSKU`, `TelaAdicionarProdutos`: `critico = mesesCobertura < coberturaAlvo * 0,6`. Constante `0,6` (60% do alvo) hardcoded em cada um dos 3 lugares, não é uma variável nomeada compartilhada.

Cobertura projetada usa `pedidoUnidades`, convertido pra caixa fechada quando MASTER — `TelaDecisaoSKU`. **Corrigido em revisão com Felipe**: a versão anterior somava o número de caixas digitado direto ao estoque em unidades, sem converter — havia um comentário no código atribuindo isso a uma “decisão

tomada junto com o TI”, mas na revisão ficou confirmado que era divergência não-intencional (bug), não regra de negócio. Hoje usa a mesma conversão de `calcularSugestaoPedido` (`pedidoNum * embalCompra` quando MASTER), consistente nos dois lugares.

Dimensão automática de quebra (Monitoramento) — `TelaMonitoramento`, linhas 939–943. Ordem fixa: Departamento → Seção → Linha → Categoria. A tela sempre quebra pela **primeira dimensão dos 3 filtros que ainda estiver em “Todos”/“Todas”**; se as 3 (Departamento/ Seção/Linha) estiverem fixadas, só quebra por Categoria se a Seção escolhida estiver no conjunto `SECOES_COM_CATEGORIA` (que são as seções que têm pelo menos uma linha de `DADOS` com `categoria` não nula). Se não houver mais nada pra quebrar, mostra só o total.

Comparação com ano anterior, proporcional — `rotuloComparacaoAno`, linhas 299–320. Se o período selecionado é o **atual** (offset 0), a comparação é textualmente marcada “(proporcional)” e citada como indo “até [data de hoje]” — mas note: **o texto muda, o número “+6,4%” não**. Se o período é passado (offset ≠ 0), o texto diz “completo”. A data de “hoje” usada em toda essa lógica é fixa: `new Date(2026, 7, 28)` (28 de agosto de 2026) — não é `new Date()` do navegador.

Regime fiscal por modalidade — `corTributacao` / `labelTributacaoCurta`, linhas 154–163. 3 modalidades possíveis: `ST_SUBSTITUTO` (azul, “ST Substituto”), `ST_RECOLHIDO` (roxo `#7C3AED`, “ST Recolhido”), qualquer outra incluindo `“NORMAL”` (cinza, “Normal”). “Monofásico” é um badge **à parte**, concatenado quando `creditoPisCofins === 0` — é tratado como atributo independente da modalidade, não como uma 4ª modalidade.

Valor NF a partir do custo — `valorAntesDoCredito`, linha 169. `valor = custo / ((1 - creditoICMS) × (1 - creditoPisCofins))`. Ou seja, a interface **nunca lê um “valor NF” de um campo próprio** — sempre reconstrói de trás pra frente a partir do custo já calculado e das taxas de crédito do produto.

Regra de exportação Winthor — `exportarWinthor`, linha 1338. Formato de 3 colunas só: código do produto, preço unitário (2 casas decimais), quantidade — **sem cabeçalho**, sempre `.xlsx`. Comentário no código: “confirmado na documentação oficial TOTVS”. Filial/ Fornecedor/Comprador **não vão na planilha**, são digitados manualmente na tela do Winthor na hora de importar.

Um pedido salvo por fornecedor — `TelaPedidos.salvarPedidos`, linha 2793. Se o carrinho tem itens de 2+ fornecedores, o clique em “Salvar pedido(s)” cria **múltiplos** pedidos salvos automaticamente, um por fornecedor, todos com `dataCriacao: “28/08/26”` fixo e status inicial `“Rascunho”`.

Confirmação de exclusão — não existe. `excluir(id)` (linha 1742) remove o pedido salvo direto do array, sem diálogo de confirmação em nenhum lugar do protótipo.

6. Layout e design, em números

Biblioteca de estilo: Tailwind CSS (classes utilitárias inline, nenhum arquivo `.css` separado). Ícones: `lucide-react`. Nenhum design system (shadcn/MUI/etc.) — é estilo próprio, montado componente a componente.

Cores (só existe tema claro — não há dark mode):

Nome/uso	Hex	Onde é usado
Navy (marca)	<code>#375DA8</code>	Cabeçalho, botões primários, valores em destaque, badge “ST Substituto”, classe A
Vermelho (alerta/ação)	<code>#DE434B</code>	Botões de ação destrutiva/decisão, alertas ruptura e margem baixa, classe S/VEND

Nome/uso	Hex	Onde é usado
Âmbar (aviso)	#B98A2E	Status “Orçamento Enviado”, alertas baixo_giro/estoque_alto, “abaixo do mínimo”
Verde (sucesso)	#15803D	Status “Exportado”, banners de confirmação
Verde claro (botão enviado)	#16A34A	Botão “Enviado ✓” em Nível 2
Roxo (ST Recolhido)	#7C3AED	Badge de tributação quando modalidade é ST_RECOLHIDO
Cinza neutro	#6B7280	Status “Rascunho”, modalidade “Normal”, textos secundários
Cinza-oliva (sem giro)	#8A8578	Ícone/cor do alerta “sem_giro”
Amarelo (barra ano anterior)	#FBBF24	Barra do mini-gráfico de venda referente ao “ano passado”
Azul de destaque (EST+PED)	#1D4ED8	Coluna EST+PED em Pedidos
Roxo de destaque (cobertura)	#7E22CE	Coluna Cobertura/Alvo em Pedidos

Opacidade de fundo: praticamente todo “fundo claro de uma cor” no protótipo é a mesma cor com sufixo hex de opacidade — `18` (~9%) pra badges/pills, `12` (~7%) ou `0D` (~5%) pra fundos de botão sutil. Não há uma escala nomeada, é escolhido caso a caso.

Tipografia: `font-family: system-ui, -apple-system, sans-serif` (única declaração de fonte em todo o arquivo, no elemento raiz). Não há fonte customizada importada. Números sempre em `font-mono` (a fonte monoespaçada padrão do sistema, via classe Tailwind), texto em peso `font-medium` / `font-semibold` / `font-bold` conforme hierarquia. Tamanhos usados (todos em `text-[Npx]`, não a escala padrão do Tailwind): 9, 10, 10.5, 11, 12, 12.5, 13, 14, 15, 16, 17, 18px. Não há uma escala nomeada tipo “h1/h2/body” — cada componente define o tamanho que achou certo.

Espaçamento: escala Tailwind padrão (múltiplos de 4px — `p-1=4px`, `p-2=8px`, `p-3=12px`, `p-4=16px` etc.), aplicada via classes utilitárias em quase toda parte. Nenhuma customização do espaçamento base do Tailwind.

Largura do container: - App principal: mobile `max-w-[420px]`, desktop `max-w-[1400px]`. - `TelaDecisaoSKU` (Nível 2) tem seu **próprio** container, com larguras diferentes: mobile `max-w-[420px]` (igual), desktop `max-w-[1100px]` (menor que o resto do app). - Sempre `mx-auto` com `border-x border-gray-200` — o conteúdo fica centralizado na tela do navegador com bordas visíveis, simulando um “cartão” mesmo em telas muito largas.

Responsivo — como funciona de fato: não existe breakpoint de CSS/media query em lugar nenhum do protótipo. A troca mobile ↔ desktop é um **estado booleano manual** (`modoDesktop`), alternado por dois botões (ícones `Smartphone/Monitor`) no cabeçalho. Cada tela tem, literalmente, dois blocos de JSX — um `if (modoDesktop) { return (...) }` inteiro, e um `return (...)` de fallback pro mobile — quase sem reaproveitamento de marcação entre os dois. Isso significa: **toda mudança de campo/coluna precisa ser feita duas vezes**, uma em cada bloco, e as duas versões já divergem em pequenos detalhes hoje (ver seção 8).

Grade: não há sistema de grid nomeado — usa `grid grid-cols-N` do Tailwind pontualmente (ex. `grid-cols-3` nos KPIs, `grid-cols-2` nos blocos de preço do Nível 2 desktop). Larguras de coluna de tabela são majoritariamente automáticas, com `min-w-[Npx]` pontual em colunas de texto longo (nome do produto,

tipicamente `min-w-[220px]` a `min-w-[260px]`) e `style={{ minWidth: Npx }}` inline em células com conteúdo empilhado (custo/valor, novo preço).

Componentes reutilizáveis:

Componente	Variações/estados	Onde é usado
<code>Segmented</code>	ativo (fundo navy, texto branco) / inativo (texto cinza)	Período, Métrica, janela de tendência
<code>classeChip</code>	A (navy) / B/C (cinza) / S/VEND (vermelho)	toda linha de produto
<code>badgeStatus</code>	4 cores por status do pedido	Pedidos Salvos
<code>badgeComNumero</code>	cor por tipo de alerta, com ou sem detalhe numérico	Alertas
<code>CenarioCard</code>	com ou sem badge “CENÁRIO REAL”	Nível 2
<code>FaixaAviso</code>	cor/ícone por tipo (sazonalidade âmbar, sucessão navy, campanha vermelho)	Nível 2
<code>SeletorPeriodo</code>	modo exibição (rótulo + setas) / modo edição (input nativo de data/semana/mês/ano)	Monitoramento, Entradas
<code>SeletorOrdenacao</code>	select simples	Pedidos, Precificação
Botões primários	fundo sólido (navy/vermelho), texto branco, <code>disabled:opacity-30</code> quando aplicável	em toda parte
Botões secundários	borda + texto colorido, sem fundo	“voltar”, “cancelar”, “desfazer status”
Inputs numéricos	borda cinza, texto centralizado, <code>font-mono</code>	toda quantidade/preço editável
Selects	aparência customizada (<code>appearance-none</code> + ícone <code>ChevronDown</code> posicionado manualmente)	todo filtro

Não há estado de **foco**, **hover** ou **desabilitado** tratado explicitamente na maioria dos componentes — só o padrão do navegador, exceto `disabled:opacity-30` (Tailwind) nos botões de ação principal quando a condição não é atendida (carrinho vazio, nada alterado etc.), e `hover:bg-gray-50` nas linhas de tabela desktop.

7. Fluxo e navegação

Não há biblioteca de rotas. Toda navegação é troca de `useState` no componente raiz (`PainelCedep`) ou dentro de cada tela. Nenhuma URL reflete o estado atual.

Mapa de navegação:

```

PainelCedep (raiz)
├─ area="painel", aba="alertas"      → TelaAlertas
├─ area="painel", aba="monitoramento" → TelaMonitoramento
├─ area="painel", aba="entradas"      → TelaEntradas
├─ area="pedidos"                    → TelaPedidos
├─   └─ onAbrirDecisao(p) ───────────┐
├─ area="pedidos_salvos"              → TelaPedidosSalvos
├─   └─ setImpressao(p) → ComprovanteImpressao (overlay, cobre a tela toda)
├─   └─ setItemAberto(p) → TelaDetalhePedidoSalvo
├─       └─ setAdicionando(true) → TelaAdicionarProdutos (substitui a tela, não é overlay)
├─ area="precificacao"                → TelaPrecificacao
├─   └─ onAbrirDecisao(p) ───────────┐
├─ skuAberto = produto ───────────┐
├─   └─ → TelaDecisaoSKU (substitui QUALQUER tela acima, é um `if` antes de tudo no componente
raiz)
├─   └─ onVoltar() → skuAberto = null → volta pra tela que estava embaixo

```

O que é passado entre telas: sempre o objeto produto inteiro (de `PRODUTOS`), não um id — `onAbrirDecisao={setSkuAberto}` e o clique passa `p` direto. Isso significa que `TelaDecisaoSKU` nunca busca o produto de novo, trabalha com a referência que recebeu no momento do clique — se o mesmo produto for reaberto depois de algo mudar em `PRODUTOS` (o que não acontece no protótipo, já que `PRODUTOS` é `const`), a tela não atualizaria sozinha.

Recarregar a página no meio do caminho: como não há persistência nenhuma (nem `localStorage`, nem backend — ver seção 8), recarregar em qualquer ponto volta o app inteiro pro estado inicial: `area="painel"`, `aba="alertas"`, `modoDesktop=false`, `pedidosSalvos=[]`. Qualquer pedido salvo, qualquer preço digitado, qualquer edição em andamento — tudo se perde, sem aviso.

Botão “voltar” do navegador: não tratado. Como não há mudança de URL, o botão voltar do navegador sai do app inteiro (ou não faz nada, dependendo de como foi carregado), não volta um passo de navegação interna do protótipo.

8. O que está faturado como “de mentira”

Sem constrangimento, listado tudo que achei:

- **Todo o app é um array em memória.** Não existe requisição de rede, `fetch`, API, nem simulação de latência. Toda “busca” é `.filter()` / `.map()` síncrono num array de 6 a 26 itens, dependendo da tela.
- **Nada persiste.** `pedidosSalvos` é `useState([])` no componente raiz — recarregar a página apaga tudo. Preços digitados em Precificação, pedido digitado em Nível 2, filtros selecionados — tudo é estado de componente, morre ao trocar de tela (Nível 2) ou ao recarregar (tudo o resto).
- **Botão “Enviar pra Cássio/Gabriela” (Nível 2) não envia nada.** Só muda o próprio estado visual (`enviado = true`), sem chamada, sem e-mail, sem notificação — e nem desabilita depois de clicado, dá pra clicar de novo indefinidamente.
- **O comparativo “+6,4%” (Monitoramento) é fixo.** Aparece do mesmo jeito não importa filtro, período, ou produto — só o texto ao lado (explicando com o que está comparando) muda de verdade. **Confirmado com Felipe:** no sistema real, isso deve virar um cálculo período-a-período genuíno assim que existir dado histórico de verdade pra comparar — não é comportamento pretendido, é limitação de não ter esse dado no protótipo.
- **O filtro de período em Monitoramento e Entradas não filtra o dado.** A UI deixa escolher Dia/Semana/Mês/Ano e navegar no tempo, mas `DADOS`, `TOP_PRODUTOS_GERAL` e `ENTRADAS` são arrays estáticos que não têm dimensão de tempo real — o número mostrado é sempre o mesmo, o período selecionado não entra em nenhuma fórmula de filtro desses três arrays.

- **“Hoje” é uma data fixa no código:** `new Date(2026, 7, 28)` — 28 de agosto de 2026, sexta-feira. Não é a data real do dispositivo. Todo cálculo de “essa semana”/“esse mês”/ navegação de período usa essa data fixa como referência.
- **dataCriacao de todo pedido salvo é a string fixa “28/08/26”**. Não usa a data real nem a data fixa do `HOJE` de forma programática — é texto hardcoded na função `salvarPedidos`.
- **Exportação de PDF não gera PDF.** `ComprovanteImpressao` monta um HTML formatado e chama `window.print()` do navegador — o PDF só existe se a pessoa escolher “Salvar como PDF” no diálogo de impressão nativo.
- **Botões “PDF”/“Excel”/“Exportar Winthor” não têm estado de carregamento nem de erro.** Se o download falhar (função `baixarArquivo`), a única resposta é um `alert()` nativo do navegador com o texto do erro — não há retry, não há notificação in-app.
- **Sem confirmação de exclusão.** “Excluir” em Pedidos Salvos remove na hora, sem diálogo.
- **Sem confirmação de saída com edição pendente.** Sair de `TelaDetalhePedidoSalvo` sem clicar “Salvar alterações” descarta silenciosamente qualquer mudança feita.
- **TelaMonitoramento não tem navegação pra produto.** É a única tela de listagem de produto que não tem `onAbrirDecisao` — clicar numa linha não faz nada.
- **Seção/Categoria/Linha, em toda parte do protótipo, são marcadas no próprio código como “ainda ilustrativas”** — não são dado real do Winthor hoje, foram inventadas pra dar estrutura de navegação/filtro. **Confirmado com Felipe:** o campo já existe de verdade no Winthor, e está em fase de cadastro. Fica a cargo do TI localizar de onde puxar esse dado no banco (Felipe não vai sugerir onde, por entender que o TI tem mais conhecimento do schema do que ele).
- **isMasterFornecedor /lista de 5 fornecedores MASTER está duplicada 6 vezes** como array literal inline em vez de sempre chamar a função nomeada — funciona hoje porque os 6 arrays são idênticos, mas é um ponto de manutenção frágil (ver seção 9).
- **Nenhuma tela trata “sem permissão”.** Não existe conceito de usuário/login/perfil em lugar nenhum do protótipo — é assumido que quem abre o app pode ver e editar tudo.
- **Nenhuma tela trata “carregando” ou “erro de rede”.** Consequência direta de não haver rede nenhuma.
- **Assimetrias entre mobile e desktop que já existem hoje** (cada uma é um “de mentira” específico, porque as duas versões foram escritas por igual mas divergiram):
 - A nota de aviso quando `filtroStatus !== "Ativo"` (Alertas) só existe no mobile.
 - A nota “‘Oficial’ não existe no varejo” (Precificação) só existe no mobile.
 - A legenda de tributação (o parágrafo explicando os regimes validados) aparece nas duas versões, mas com textos diferentes — a versão desktop lista os regimes um a um, a mobile só diz que foram “validados item a item contra a V10 real”.

9. Decisões que eu tomei

Coisas que o código evidencia como escolha deliberada (com nota própria no código) ou como caminho escolhido entre várias opções possíveis, mesmo sem explicação do porquê:

- **Cobertura projetada usa `pedidoUnidades`, convertido pra caixa fechada.** Uma versão anterior deste protótipo usava `pedidoNum` direto (sem converter), com um comentário atribuindo isso a uma “decisão tomada junto com o TI” — em revisão com Felipe, ficou confirmado que era uma divergência não-intencional, não uma regra de negócio real. Foi corrigida pra usar a mesma conversão de `calcularSugestaoPedido`.
- **`custoGerencial` foi definido como sinônimo do custo em cenário “Oficial”,** e não como um 4º valor independente. Ou seja, sempre que a interface precisa de “custo no cenário Oficial”, ela lê `custoGerencial`.

direto — essa equivalência é uma decisão de modelagem do protótipo, confirmada contra os 8 casos reais, mas é importante que o TI saiba que são **o mesmo campo**, não dois campos que coincidentemente batem.

- **A inversão de qual par de cenários é igual, conforme a modalidade** (ST_SUBSTITUTO: Sem Redução = Oficial; NORMAL: Sem Redução = ST s/Valor) foi implementada como um `if` dentro de `custoPorCenario` e `margemAtualPorCenario` — batia nos 8 casos reais testados, mas era uma regra só inferida por comparação empírica de números, sem explicação documentada. **Explicação confirmada com Felipe** (ver seção 5): produtos com ST não recebem redução de alíquota de ICMS — só os Normais — por isso Oficial e Sem Redução colapsam no mesmo valor pra ST_SUBSTITUTO (não existe uma variante “reduzida” separada pra esse regime).
- **Varejo nunca tem cenário “Oficial”** — decisão explícita (comentário: “exclusivo filiais 02/09”), implementada simplesmente omitindo a opção do array de cenários de varejo, sem nenhuma lógica condicional por filial (não existe conceito de filial em lugar nenhum do protótipo).
- **Um pedido salvo por fornecedor**, nunca misto — decisão espelhando uma exigência real do Winthor (rotina 220), documentada em comentário como “confirmado na documentação oficial TOTVS”.
- **O card do produto usa o objeto inteiro, não o código**, ao navegar pra Nível 2 — escolha de simplicidade (evita um lookup extra), mas significa acoplamento entre a tela de origem e a tela de destino via a mesma referência de objeto.
- **Cobertura crítica em 60% do alvo** (* 0.6) é repetida como número mágico em 3 lugares diferentes, sempre o mesmo valor — parece uma regra de negócio real (limiar de alerta visual), mas não está nomeada nem centralizada.
- **Formato Winthor sem cabeçalho, 3 colunas, sem filial/fornecedor/comprador** — decisão bem documentada no comentário do código como vinda da documentação oficial da TOTVS, a mais bem justificada de todo o protótipo.
- **Status do pedido segue uma máquina de estados linear e reversível** (Rascunho ↔ Orçamento Enviado ↔ Fechado ↔ Exportado, sempre um passo por vez, com “desfazer” simétrico) — não existe caminho pra pular etapas nem pra voltar mais de um passo de uma vez.

Perguntas em aberto

O que eu não consegui determinar só olhando o código — fica registrado como dúvida, não como suposição preenchida:

Todas as dúvidas originalmente listadas aqui (comparativo “+6,4%”, origem dos fatores de prazo, a regra de “Sem Redução” por modalidade, o regime fiscal de CAR 80/3M, a divergência de `MESES_EST+PED`, e a origem de Seção/Categoria/Linha) **foram revisadas e resolvidas em conversa direta com Felipe** — as respostas já estão incorporadas nas seções 5, 8 e 9 acima (marcadas como “Confirmado com Felipe” ou “Corrigido em revisão com Felipe”), e uma delas (`MESES_EST+PED`) já foi corrigida no próprio código do protótipo.

Não ficou nenhuma pergunta em aberto até o momento. Se a equipe de TI, ao ler este documento, tiver dúvidas novas, elas devem ser registradas aqui numa próxima revisão.